

A Regularized Convolutional Neural Network for Semantic Image Segmentation

Fan JIA*, Jun LIU†, and Xue-cheng TAI*

Abstract. Convolutional neural networks (CNNs) show outstanding performance in many image processing problems, such as image recognition, object detection and image segmentation. Semantic segmentation is a very challenging task that requires recognizing, understanding what's in the image in pixel level. Though the state of the art has been greatly improved by CNNs, there is no explicit connections between prediction of neighbouring pixels. That is, spatial regularity of the segmented objects is still a problem for CNNs. In this paper, we propose a method to add spatial regularization to the segmented objects. In our method, the spatial regularization such as total variation (TV) can be easily integrated into CNN network. It can help CNN find a better local optimum and make the segmentation results more robust to noise. We apply our proposed method to Unet and Segnet, which are well established CNNs for image segmentation, and test them on WBC, CamVid and SUN-RGBD datasets, respectively. The results show that the regularized networks not only could provide better segmentation results with regularization effect than the original ones but also have certain robustness to noise.

Key words. CNN, Image Segmentation, Regularization.

AMS subject classifications. 68U10, 68T10, 65K10

1. Introduction. Convolutional Neural Networks (CNNs) [17] have achieved prominent performance in a series of image processing tasks, such as image classification [12, 15, 31], object detection [8, 9, 12, 18, 22, 25] and image segmentation [1, 7, 19, 23, 32]. CNNs which dwarf systems relying on hand-crafted features use millions of parameters to learn latent features from large scale training datasets. It is intractable to design a hand-crafted feature which allows to learn increasingly abstract data representations. However, CNNs could do it well [31].

Image segmentation is a process of segmenting a digital image into different regions. It aims to simplify and/or change the representation of an image into something that is more meaningful and easier to analyze. Semantic image segmentation is a much more challenging segmentation task. It requires understanding the image in pixel level. More precisely, it is a process of assigning a label to every pixel in an image such that pixels with the some label share some common features [2, 26].

Currently, CNNs regard semantic image segmentation as a dense prediction problem. They predict the classification of each pixel independently. Though the convolution kernels are shareable and predictions to different pixels have implicit connection. There is no explicit connection when predicting adjacent pixels which is not accord with fact that classifications of pixels inside an object are interdependent. That is, spatial regularization is still missing. Though many efforts have been made and the accuracy has been continuously improved,

*Department of Mathematics, Hong Kong Baptist University (jiafan@life.hkbu.edu.hk, xuecheng-tai@hkbu.edu.hk).

†Laboratory of Mathematics and Complex Systems (Ministry of Education of China), School of Mathematical Sciences, Beijing Normal University (jliu@bnu.edu.cn).

CNNs fail to provide segmentation results with regularization effect. This is because the popular CNNs are usually continuous mappings which are composite mappings of continuous operators such as affine transformations and continuous activation functions (e.g. Soft-max, ReLU, Sigmoid). They can not provide spatial regularization for the segmented objects. As shown in Figure 1, we trained the original Unet [23] and our RUnet on White Blood Cell (WBC) Dataset [33]. To test the robustness of the two methods with respect to noise, we test the two trained network on image with added noise. The segmentation results of the original Unet [23] becomes much worse, though the noise level is not high. Nevertheless, RUnet can still achieve good segmentation result.

Image restoration such as denoising and deblurring are the most fundamental tasks in image processing. It is important but difficult to preserve image structures (such as edges) in image restoration [30]. Total Variation (TV) method shows good performance when handling minimizing problems in image restoration [4, 5, 24], since it can preserve discontinuity. We shall propose a way to add spatial TV regularization to CNNs.

Our essential idea is to add spatial regularization to activation functions. In this paper, we focus on applying spatial regularization to softmax. The same technique can also be applied to other activation functions. This gives us CNNs with spatial regularity.

We apply our proposed method to Unet [23], Segnet [1] and evaluate their performance on WBC Image Dataset [33], CamVid Dataset [13] and SUN-RGBD Dataset [28]. Unet is well known for its outstanding performance on biomedical image segmentation. It achieved the first place in ISBI cell tracking challenges 2015 leading other methods by a large margin. Segnet achieves better performance on real world scenes such as CamVid and SUN-RGBD datasets than DeepLab-LargeFOV [6], FCN [19], DeconvNet [20]. WBC Dataset consists of color cell images, which are collected from the CellaVision blog. The cell images allow us to observe distinct object details. CamVid Dataset and SUN-RGBD Dataset are much more complex datasets which consist of real world road scenes and indoor scenes, respectively. They are chosen as benchmark of Segnet.

Unet with regularized softmax (RUnet) and Segnet with regularized softmax (RSegnet) achieve better performance than original Unet and Segnet on testing datasets. The segmentation result is spatially regularized and robust. Our approach gives a promising direction for semantic segmentation tasks, which may benefit a series of CNN-based image processing tasks. Our main contributions are in the following:

- We propose a framework to integrate the traditional variational regularization method into deep convolutional neural networks. In this work, we present it for the softmax activation function. The same idea can be applied to other activation functions. It is known that spatial regularization is important for image and vision problems. So far, it is still missing to have good spatial regularization effects for these applications.
- When spatial regularization is added to CNNs, one essential difficulty is how to find a simple and clear way to calculate the gradient decent direction for general loss functions. We first give the general formula of computing gradients when integrate total variation to CNN. Then we propose an efficient method which needs very little modifications to existing CNNs and their numerical implementations, but has very visible regularization effects with much better robustness to noise and good improved accuracy.

- We give experiments of our proposed method on two CNNs for image segmentation, i.e. Unet and Segnet. By testing them on three datasets, it is numerically verified that the new method could produce smoother objects and has better robustness to noise.

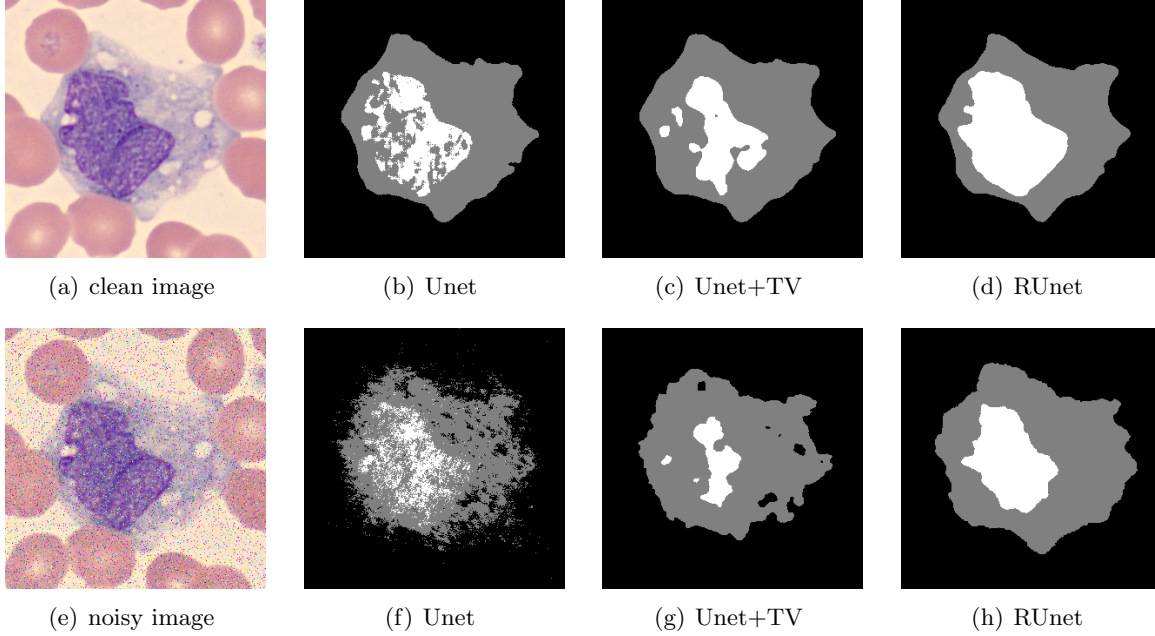


Figure 1. An example of segmentation results by performing the original Unet [23] and our proposed regularized Unet (RUnet) on WBC Dataset[33]. When adding noise to image, the segmentation of nucleus by Unet becomes messy (Figure 1(f)). If we add post-processing to the prediction of Unet (Unet+TV), the result is still not desirable (Figure 1(c), Figure 1(g)). However, our proposed RUnet could provide smooth segmentation result (Figure 1(d), Figure 1(h)).

The paper is organized as follows. In Section 2, we give brief descriptions to related work, general neural network for semantic image segmentation and total variation, respectively. Our proposed method is given in Section 3. In this section, we apply our proposed method to softmax layer and give the general formulas for forward propagation, backward propagation. Some implementation details are also illustrated here. The experimental results are described in Section 4, and the conclusions follow in Section 5.

2. Related Work. Semantic segmentation task has long been an attractive topic in image processing. In early years, systems relying on hand-crafted features are combined with classifiers, such as Boosting [16, 29], Random Forests [3, 27], or Support Vector Machines [3]. These methods often use region based method to predict the probability of each pixel. However, the choice of hand-crafted features can be very crucial, the performance of same feature can vary much when applied to different kinds of datasets. Meanwhile, the performance of such systems is compromised by insufficient feature representation ability.

R-CNN [9] and SDS [10] use CNN as feature extractor which followed by final refinement step to help improve segmentation. Nevertheless, for pixel-wise semantic segmentation

problems, the region based approach becomes bottleneck.

FCN [19] is a successful attempt training end-to-end, pixel-to-pixel convolutional network on semantic segmentation. It achieved 30% relative improvement compared with previous best PASCAL VOC 11/12 test results.

After FCN, a series of CNNs come out to improve the segmentation performance, such as Unet [23], PSPNet [32], Segnet [1] and Deeplab [7]. Unet fuses high level feature map with low level feature map and shows prominent performance in medical image processing. PSPNet exploits the capability of global context information by different-region-based context aggregation. Segnet uses the max pooling indices to upsample (without learning) the feature map(s) and convolves with a trainable decoder filter bank. Deeplab applies atrous convolution for dense feature extraction and enlarge the field-of-view.

Some CNNs try to boost their ability to capture fine details by employing a fully-connected Conditional Random Field (CRF) [14]. However they didn't fully integrate CRF into CNN, thus CRF didn't contribute to updating the weights. Technique has also been proposed to regularize the parameter set to align results to edges [21]. Nevertheless, no one has tried to regularize segmentation results by adding spatial regularization to activation functions. Next, we will review the general CNN for semantic image segmentation and TV regularization.

2.1. General Neural Network for Semantic Image Segmentation. Let $v \in \mathbb{R}^{N_1 N_2}$ be a column vector by stacking the columns of image with size $N_1 \times N_2$. Taking v as an input of a pixel-wise segmentation neural network. Mathematically, this network can be written as a parameterized nonlinear operator $\mathcal{N}_\Theta$ defined by $v^K = \mathcal{N}_\Theta(v)$. The output v^K of the network is given by the following recursive connections

$$(2.1) \quad \begin{cases} v^0 = v, \\ v^k = \mathcal{A}^k(\mathcal{T}_{\Theta^{k-1}}(v^{k-1})), k = 1, \dots, K, \end{cases}$$

where $\mathcal{A}^k$ is an activation function (e.g. sigmoid, softmax, ReLU) or sampling (e.g. down-sampling, up-sampling, dilated convolution) operator or their compositions and $\mathcal{T}_{\Theta^{k-1}}$ is often chosen as an affine transformation with the representation $\mathcal{T}_{\Theta^{k-1}}(v) = \mathcal{W}^{k-1}v + b^{k-1}$, in which $\mathcal{W}^{k-1}, b^{k-1}$ are linear operator (e.g. convolution) and translation, respectively. The parameter set is $\Theta = \{\Theta^k = (\mathcal{W}^k, b^k) | k = 0, \dots, K-1\}$. The output of this network $v^K \in \{0, 1\}^{C \times N_1 N_2}$ should be a binary classification matrix whose c -th column is a binary characteristic vector for c -th class, $c = 1, 2, \dots, C$. By giving M images $\mathcal{V} = (v_1, v_2, \dots, v_M) \in \mathbb{R}^{M \times N_1 N_2}$ and their ground truth segmentation $\mathcal{U} = \text{stack}(\mathcal{U}_1, \mathcal{U}_2, \dots, \mathcal{U}_M) \in \{0, 1\}^{M \times C \times N_1 N_2}$ with $\mathcal{U}_m \in \{0, 1\}^{C \times N_1 N_2}$, the training process is to learn a parameter set Θ which minimizes a loss functional $\mathcal{L}(\mathcal{N}_\Theta(\mathcal{V}), \mathcal{U})$, namely

$$(2.2) \quad \Theta^* = \arg \min_{\Theta} \mathcal{L}(\mathcal{N}_\Theta(\mathcal{V}), \mathcal{U}).$$

In many references, the loss functions are set to be the cross entropy which is given by

$$(2.3) \quad \mathcal{L}(\mathcal{N}_\Theta(\mathcal{V}), \mathcal{U}) = -\frac{1}{M} \sum_{m=1}^M \langle \mathcal{U}_m, \log \mathcal{N}_\Theta(v_m) \rangle.$$

The algorithm of learning is a gradient descent method:

$$(2.4) \quad (\Theta^k)^{step} = (\Theta^k)^{step-1} - \tau_\Theta \frac{\partial \mathcal{L}}{\partial \Theta^k} \Big|_{\Theta^k = (\Theta^k)^{step-1}},$$

where $k = 0, \dots, K-1, step = 1, 2, \dots$ is the iteration number and τ_Θ is a time step or so called learning rate. $\frac{\partial \mathcal{L}}{\partial \Theta^k}$ can be calculated by backpropagation technique using chain rule. Denoting $o^k = \mathcal{T}_{\Theta^{k-1}}(v^{k-1})$, then Equation (2.1) becomes

$$(2.5) \quad \begin{cases} v^k = \mathcal{A}^k(o^k), \\ o^k = \mathcal{T}_{\Theta^{k-1}}(v^{k-1}), \end{cases}$$

where $k = 1, \dots, K$. Let us write $\Delta^k = \frac{\partial \mathcal{L}}{\partial o^k}$, then the backpropagation scheme becomes

$$(2.6) \quad \begin{cases} \Delta^k = \frac{\partial v^k}{\partial o^k} \cdot \frac{\partial o^{k+1}}{\partial v^k} \cdot \frac{\partial \mathcal{L}}{\partial o^{k+1}} \\ = \frac{\partial \mathcal{A}^k}{\partial o^k} \cdot \frac{\partial \mathcal{T}_{\Theta^k}}{\partial v^k} \cdot \Delta^{k+1}, \\ \frac{\partial \mathcal{L}}{\partial \Theta^k} = \frac{\partial o^{k+1}}{\partial o^k} \cdot \frac{\partial \mathcal{L}}{\partial o^{k+1}} = \frac{\partial \mathcal{T}_{\Theta^k}}{\partial o^k} \cdot \Delta^{k+1}, \end{cases}$$

where $k = 0, 1, \dots, K-1$.

2.2. Total Variation. The total variation (TV) is proposed to produce piece-wise constants cartoon restorations in ROF model [24]. TV can be written as

$$(2.7) \quad \text{TV}(u) = \int_{\Omega} |\nabla u(x)| dx,$$

where Ω is a bounded subset of $\mathbb{R}^2$, u is a single channel image. It has a dual formulation as

$$(2.8) \quad \text{TV}(u) = \sup_{\xi \in \mathbb{B}} \left\{ \int_{\Omega} u(x) \text{div} \xi(x) dx \right\},$$

where $\mathbb{B} = \{\xi \in C_0^1(\Omega; \mathbb{R}^2) \mid \|\xi\|_{\infty} = \max_{x \in \Omega} \{\|\xi(x)\|_2\} \leq 1\}$.

When u has multi-channels, we sum up the contributions of the separate channels, and the definition is given by the following:

$$(2.9) \quad \text{TV}(\mathbf{u}) = \sum_{c=1}^C \int_{\Omega} |\nabla u_c(x)| dx$$

where C is the number of channels. The dual formulation is given by:

$$(2.10) \quad \text{TV}(\mathbf{u}) = \sup_{\xi_1, \dots, \xi_C \in \mathbb{B}} \left\{ \sum_{c=1}^C \int_{\Omega} u_c(x) \text{div} \xi_c(x) dx \right\}.$$

For discrete TV and the related dual formulation, they have the similar expressions [4].

3. Proposed Method.

3.1. Intuition. Usually, a CNN contains dozens of activation functions and softmax function is the most commonly used in the last layer. Softmax function is a function that takes as input a vector of C real numbers, and normalizes it into a probability distribution consisting of C probabilities.

In fact, softmax could be derived from a minimization problem. When given $\mathbf{o} \in \mathbb{R}^{C \times N_1 N_2}$ as the input, C is the number of classes, $N_1 \times N_2$ is the image size, we want to find a corresponding output $\mathcal{A} \in \mathbb{R}^{C \times N_1 N_2}$ such that $\mathcal{A}$ is the minimizer of the following problem:

$$(3.1) \quad \begin{aligned} & \min - \langle \mathcal{A}, \mathbf{o} \rangle + \langle \mathcal{A}, \log \mathcal{A} \rangle, \\ & s.t. \sum_c^C \mathcal{A}_{ci} = 1, \forall i = 1, \dots, N_1 N_2. \end{aligned}$$

Let $\mathcal{A} = (\mathcal{A}_1, \dots, \mathcal{A}_C)$, $\mathcal{A}_c \in \mathbb{R}^{N_1 N_2}$ for $c = 1, 2, \dots, C$. Some simple calculations can show that the minimizer of the above problem is:

$$(3.2) \quad \hat{\mathcal{A}}^*_{\cdot j} = \frac{\exp(\mathbf{o}_j)}{\sum_{c=1}^C \exp(\mathbf{o}_c)}, j = 1, \dots, C.$$

$\hat{\mathcal{A}}^*_{\cdot j}$ is the j -th class probability map of the input image. One can easily see that this is just the commonly used softmax activation function, i.e.

$$(3.3) \quad \hat{\mathcal{A}}^* = \text{Softmax}(\mathbf{o})$$

However, this function doesn't have any spatial regularization. Prediction of each pixel is independent of other pixels.

3.2. Proposed Regularized Softmax Layer. Inspired by the softmax variational problem Equation (3.1), we propose to replace the softmax function by the following regularized softmax:

$$(3.4) \quad \begin{aligned} & \min - \langle \mathcal{A}, \mathbf{o} \rangle + \langle \mathcal{A}, \log \mathcal{A} \rangle + \lambda TV(\mathcal{A}), \\ & s.t. \sum_c^C \mathcal{A}_{ci} = 1, \forall i = 1, \dots, N_1 N_2, \end{aligned}$$

where λ is the regularization parameter which controls the regularization effect. According to the definition of multi-channels' total variation Equation (2.9) and Equation (2.10), we have

$$(3.5) \quad TV(\mathcal{A}) = \sum_{c=1}^C \int_{\Omega} |\nabla \mathcal{A}_c(x)| dx = \sup_{\xi_1, \dots, \xi_C \in \mathbb{B}} \left\{ \sum_{c=1}^C \int_{\Omega} \mathcal{A}_c(x) \text{div} \xi_c(x) dx \right\}$$

Compared to the traditional neural network Equation (2.5), the activation function $\mathcal{A}$ is replaced by the solution of a TV regularized minimization problem. This is significantly different from the existing continuous neural network mappings. Moreover, the problem in Equation (3.4) can be easily solved by primal-dual method:

$$(3.6) \quad \begin{aligned} (\tilde{\mathcal{A}}^*, \boldsymbol{\eta}^*) &= \arg \min_{\mathcal{A}} \max_{\xi \in \mathbb{B}} \{ - \langle \mathcal{A}, \mathbf{o} \rangle + \langle \mathcal{A}, \log \mathcal{A} \rangle + \lambda \langle \mathcal{A}, \text{div} \xi \rangle \}, \\ & s.t. \sum_c^C \mathcal{A}_{ci} = 1, \forall i = 1, \dots, N_1 N_2. \end{aligned}$$

Similar to the Chambolle type projection algorithm [4], the solutions of the above min-max problem satisfies the following relationship:

$$(3.7) \quad \tilde{\mathcal{A}}_j^* = \frac{\exp(\mathbf{o}_j - \lambda \text{div} \boldsymbol{\eta}_j^*)}{\sum_{c=1}^C \exp(\mathbf{o}_c - \lambda \text{div} \boldsymbol{\eta}_c^*)}, j = 1, \dots, C.$$

We can use the following primal-dual gradient algorithm to find the solution in an iterative way:

$$(3.8) \quad \begin{cases} \boldsymbol{\xi}^{t+1} &= \boldsymbol{\xi}^t - \tau \lambda \nabla \mathcal{A}^t, \\ \boldsymbol{\eta}^{t+1} &= \mathcal{P}_{\mathbb{B}}(\boldsymbol{\xi}^{t+1}), \\ \mathcal{A}^{t+1} &= \mathcal{S}(\mathbf{o} - \lambda \text{div}(\boldsymbol{\eta}^{t+1})), \end{cases}$$

where $\mathcal{S}$ is the softmax operator, t is the iteration number and τ is a time step, $\mathcal{P}_{\mathbb{B}}$ is a projection operator onto the convex set $\mathbb{B}$, given $\mathbf{y} = (y_1, y_2) = \boldsymbol{\xi}_{cj}$, $\mathcal{P}_{\mathbb{B}}(\mathbf{y})$ is defined by

$$(3.9) \quad \mathcal{P}_{\mathbb{B}}(\mathbf{y}) = \begin{cases} \mathbf{y}, & \text{if } \|\mathbf{y}\|_2 \leq 1 \\ \frac{\mathbf{y}}{\|\mathbf{y}\|}, & \text{if } \|\mathbf{y}\|_2 > 1 \end{cases}$$

So $\mathcal{P}_{\mathbb{B}}(\boldsymbol{\xi})$ refers to project every $\boldsymbol{\xi}_{cj} \in \boldsymbol{\xi}$ onto $\mathbb{B}$. Mathematically, $\boldsymbol{\eta}^* = \lim_{t \rightarrow +\infty} \boldsymbol{\xi}^{t+1}$ and $\tilde{\mathcal{A}}^* = \lim_{t \rightarrow +\infty} \mathcal{A}^{t+1}$. In real computation, when the iteration Equation (3.8) converges, we can get $\tilde{\mathcal{A}}^*$ and $\boldsymbol{\eta}^*$.

Thus, given $\mathbf{o}$ as the input of regularized softmax layer, we perform Equation (3.8) to obtain a convergent $\tilde{\mathcal{A}}$ and $\boldsymbol{\eta}$, then the new regularized activation function has the following simple expression:

$$(3.10) \quad \tilde{\mathcal{A}} = \text{Softmax}(\mathbf{o} - \lambda \text{div} \boldsymbol{\eta}) := \mathcal{S}(\mathbf{o} - \lambda \text{div} \boldsymbol{\eta}).$$

3.3. Regularized ReLU Layer. Our proposed method could bring regularization effect to the segmentation results and the similar idea could be easily applied to other activation functions.

For example, the popular ReLU activation function is exactly the solution to the following minimization problem:

$$(3.11) \quad \text{ReLU}(\mathbf{o}) = \arg \min_{\mathcal{A} \geq 0} \left\{ \frac{1}{2} \|\mathbf{o} - \mathcal{A}\|_2^2 \right\}.$$

Then the regularized ReLU can be given by a nonnegative constraint ROF model:

$$(3.12) \quad \arg \min_{\mathcal{A} \geq 0} \left\{ \frac{1}{2} \|\mathbf{o} - \mathcal{A}\|_2^2 + \lambda \text{TV}(\mathcal{A}) \right\}.$$

Similar to the regularized softmax, the problem in Equation (3.12) can be solved as follows:

$$(3.13) \quad (\mathcal{A}^*, \boldsymbol{\eta}^*) = \arg \min_{\mathcal{A} \geq 0} \max_{\boldsymbol{\xi} \in \mathbb{B}} \left\{ \frac{1}{2} \|\mathbf{o} - \mathcal{A}\|_2^2 + \lambda \langle \mathcal{A}, \text{div} \boldsymbol{\xi} \rangle \right\}.$$

The similar primal-dual gradient algorithm could be used to find the solution in an iterative way:

$$(3.14) \quad \begin{cases} \boldsymbol{\xi}^{t+1} &= \boldsymbol{\xi}^t - \tau \lambda \nabla \mathcal{A}^t, \\ \boldsymbol{\eta}^{t+1} &= \mathcal{P}_{\mathbb{B}}(\boldsymbol{\xi}^{t+1}), \\ \mathcal{A}^{t+1} &= \max(\mathbf{0}, \mathbf{o} - \lambda \operatorname{div}(\boldsymbol{\eta}^{t+1})), \end{cases}$$

Once we get convergent $\mathcal{A}$ and $\boldsymbol{\eta}$, the new regularized ReLU function has the following simple expression:

$$(3.15) \quad \mathcal{A} = \operatorname{ReLU}(\mathbf{o} - \lambda \operatorname{div} \boldsymbol{\eta}).$$

Usually, dozens of ReLU layers are employed to process feature maps from low level to high level in a CNN. The computational burden will be extremely high if we compute convergent $\mathcal{A}$ and $\boldsymbol{\eta}$ for every active layer. In this paper, we just consider the activation function in the last layer as to be the TV regularized softmax function in image segmentation problem.

3.4. Backpropagation of Regularized Softmax. Given initial $\mathcal{A}^0$ and $\boldsymbol{\xi}^0$ in the forward propagation stage, we propagate $\mathbf{o}$ through Equation (3.8) to achieve a regularized $\mathbf{o}$. When doing backpropagation, we need to compute the gradient of loss $\mathcal{L}$ with respect to $\mathbf{o}$. Since Equation (3.8) is computed $t + 1$ iterations in the forward propagation, we compute the gradients in an inverse order.

When $k = 1, \dots, t + 1$, $\boldsymbol{\eta}^k$ is the input to compute $\mathcal{A}^k$, so we have

$$(3.16) \quad \frac{\partial \mathcal{L}}{\partial \boldsymbol{\eta}^k} = \frac{\partial \mathcal{L}}{\partial \mathcal{A}^k} \cdot \frac{\partial \mathcal{A}^k}{\partial \boldsymbol{\eta}^k}, \quad k = 1, \dots, t + 1.$$

Similarly, $\boldsymbol{\xi}^{t+1}$ is the input to compute $\boldsymbol{\eta}^{t+1}$, when $k = 1, \dots, t$, $\boldsymbol{\xi}^k$ is the input to compute $\boldsymbol{\eta}^k$ and $\boldsymbol{\xi}^{k+1}$, so we have

$$(3.17) \quad \frac{\partial \mathcal{L}}{\partial \boldsymbol{\xi}^k} = \begin{cases} \frac{\partial \mathcal{L}}{\partial \boldsymbol{\eta}^k} \cdot \frac{\partial \boldsymbol{\eta}^k}{\partial \boldsymbol{\xi}^k}, & k = t + 1 \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\eta}^k} \cdot \frac{\partial \boldsymbol{\eta}^k}{\partial \boldsymbol{\xi}^k} + \frac{\partial \mathcal{L}}{\partial \boldsymbol{\xi}^{k+1}}, & k = 1, \dots, t. \end{cases}$$

As well, when $k = 0, \dots, t$, $\mathcal{A}^k$ is the input to compute $\boldsymbol{\xi}^{k+1}$, so we have

$$(3.18) \quad \frac{\partial \mathcal{L}}{\partial \mathcal{A}^k} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\xi}^{k+1}} \cdot \frac{\partial \boldsymbol{\xi}^{k+1}}{\partial \mathcal{A}^k}, \quad k = 0, \dots, t.$$

When $k = 0, \dots, t + 1$, $\mathbf{o}$ is the input to compute $\mathcal{A}^k$. Given $\mathcal{A}^0 = \mathcal{S}(\mathbf{o})$, we have

$$(3.19) \quad \frac{\partial \mathcal{L}}{\partial \mathbf{o}} = \frac{\partial \mathcal{L}}{\partial \mathcal{A}^0} \cdot \mathcal{S}'(\mathbf{o}) + \sum_{k=1}^{t+1} \frac{\partial \mathcal{L}}{\partial \mathcal{A}^k} \cdot \mathcal{S}'(\mathbf{o} - \lambda \operatorname{div}(\boldsymbol{\eta}^k)).$$

During the backpropagation stage, $\frac{\partial \mathcal{L}}{\partial \mathcal{A}^{t+1}}$ could be given by the loss layer, so we can iteratively obtain $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\eta}^{t+1}}, \frac{\partial \mathcal{L}}{\partial \boldsymbol{\xi}^{t+1}}, \frac{\partial \mathcal{L}}{\partial \mathcal{A}^t}, \dots, \frac{\partial \mathcal{L}}{\partial \boldsymbol{\eta}^1}, \frac{\partial \mathcal{L}}{\partial \boldsymbol{\xi}^1}, \frac{\partial \mathcal{L}}{\partial \mathcal{A}^0}$ by Equation (3.16), Equation (3.17) and Equation (3.18).

Finally, we can get $\frac{\partial \mathcal{L}}{\partial \mathbf{o}}$ by Equation (3.19).

3.5. Implementation Details. During training stage, we have to compute Equation (3.8) tens to hundreds times for each CNN iteration in order to obtain convergent $\mathcal{A}$ and $\boldsymbol{\eta}$ in the forward propagation stage. Usually, CNN needs dozens of thousands iterations to converge. It means that Equation (3.8) will be computed million times during the whole training stage, which is a huge computation burden. What's more, we have to iteratively compute $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\eta}^{t+1}}, \frac{\partial \mathcal{L}}{\partial \boldsymbol{\xi}^{t+1}}, \frac{\partial \mathcal{L}}{\partial \mathcal{A}^t}, \dots, \frac{\partial \mathcal{L}}{\partial \boldsymbol{\eta}^1}, \frac{\partial \mathcal{L}}{\partial \boldsymbol{\xi}^1}, \frac{\partial \mathcal{L}}{\partial \mathcal{A}^0}$ and keep those matrices in memory in backpropagation stage. Numerous computation and memory resources are required. The period for training one batch of models will be as long as weeks. And the mini-batch size will be smaller due to more memory is required for training each image. However, smaller mini-batch size may lead to decline in accuracy.

Currently, we compute Equation (3.8) just once for each training iteration and it will be fully performed during the testing stage. This is a trade-off between regularization effect and computation, memory resources. Though there will be less regularization effect, the demand for computation and memory resources during training stage is greatly reduced. Visible regularization effect is still observed in our experimental results in section Section 4.

In order to keep consistent with the one iteration Equation (3.8) in the forward propagation, we design a step-by-step strategy to compute $\frac{\partial \mathcal{L}}{\partial \boldsymbol{o}}$ in the backward propagation stage. In all computation, we perform Equation (3.8) just one iteration and we'll get

$$(3.20) \quad \begin{cases} \boldsymbol{\xi} &= \boldsymbol{\xi}_0 - \tau \lambda \nabla \mathcal{S}(\boldsymbol{o} - \lambda \text{div}(\boldsymbol{\eta})), \\ \boldsymbol{\eta} &= \mathcal{P}_{\mathbb{B}}(\boldsymbol{\xi}), \\ \mathcal{A} &= \mathcal{S}(\boldsymbol{o} - \lambda \text{div}(\boldsymbol{\eta})). \end{cases}$$

We set the initialization $\boldsymbol{\xi}_0$ and $\boldsymbol{\eta}_0$ to $\mathbf{0}$, respectively. Then the one iteration scheme Equation (3.20) could be simplified as :

$$(3.21) \quad \begin{cases} \boldsymbol{\xi} &= -\tau \lambda \nabla \mathcal{S}(\boldsymbol{o}), \\ \boldsymbol{\eta} &= \mathcal{P}_{\mathbb{B}}(\boldsymbol{\xi}), \\ \mathcal{A} &= \mathcal{S}(\boldsymbol{o} - \lambda \text{div}(\boldsymbol{\eta})) \end{cases}$$

Let $\mathcal{L}$ be the loss function, according to the backpropagation scheme Equation (2.6), the gradient of $\mathcal{L}$ with respect to $\boldsymbol{o}$ in Equation (3.21) is computed as:

$$(3.22) \quad \begin{aligned} \frac{\partial \mathcal{L}}{\partial \boldsymbol{o}} &= \frac{\partial \mathcal{L}}{\partial \mathcal{A}} \cdot \frac{\partial \mathcal{A}}{\partial \boldsymbol{o}}, \\ &= \frac{\partial \mathcal{L}}{\partial \mathcal{A}} \cdot (\mathcal{S}'(\boldsymbol{o} - \lambda \text{div}(\boldsymbol{\eta})) + \frac{\partial \mathcal{A}}{\partial \boldsymbol{\eta}} \cdot \frac{\partial \boldsymbol{\eta}}{\partial \boldsymbol{o}}), \\ &= \frac{\partial \mathcal{L}}{\partial \mathcal{A}} \cdot \mathcal{S}'(\boldsymbol{o} - \lambda \text{div}(\boldsymbol{\eta})) + \frac{\partial \mathcal{L}}{\partial \boldsymbol{\eta}} \cdot \frac{\partial \boldsymbol{\eta}}{\partial \boldsymbol{o}}, \end{aligned}$$

where $\frac{\partial \boldsymbol{\eta}}{\partial \boldsymbol{o}} = -\tau \lambda \frac{\partial \nabla \mathcal{S}}{\partial \boldsymbol{\xi}} \cdot \frac{\partial \mathcal{P}_{\mathbb{B}}}{\partial \boldsymbol{\xi}}.$

In Equation (3.22) we can see that $\boldsymbol{\eta}$ contributes to updating gradients during the back-propagation stage. This is quite different from other post-processing methods such as CRF.

$\frac{\partial \mathcal{L}}{\partial \boldsymbol{o}^{k+1}}$ in Equation (2.6) will be updated by Equation (3.22) for the proposed regularized network. Since the item $\tau \lambda$ in Equation (3.21) and Equation (3.22) could be seen as a scaled step size. In our implementations, we define it as a new constant and fix it by manual tuning. The regularization parameter λ will be learned as explained in the next section.

3.6. Training of the Regularization Parameter λ . The regularization parameter λ controls regularization effect. When it is too large, the output may be over-regularized, leading to a drop in accuracy. When it is too small, the TV item will contribute little to training. Generally, we manually set different λ and select a best one. However, it could be quite boring and inefficient to try different λ for each CNN on each dataset. Here, we introduce a training scheme to select λ automatically instead of manual setting. This will also help improve the training procedure.

The gradient of $\mathcal{L}$ with respect to λ is in the following:

$$(3.23) \quad \frac{\partial \mathcal{L}}{\partial \lambda} = \frac{\partial \mathcal{A}}{\partial \lambda} \cdot \frac{\partial \mathcal{L}}{\partial \mathcal{A}} = -\text{div}(\boldsymbol{\eta}) \cdot \mathcal{S}'(\boldsymbol{o} - \lambda \text{div}(\boldsymbol{\eta})) \cdot \frac{\partial \mathcal{L}}{\partial \mathcal{A}}.$$

When doing backpropagation in each iteration during training stage, we both update the parameter set Θ and λ by gradient descend method simultaneously. λ is updated as follows:

$$(3.24) \quad \lambda^{step+1} = \lambda^{step} - \tau_\lambda \left. \frac{\partial \mathcal{L}}{\partial \lambda} \right|_{\lambda=\lambda^{step}},$$

where τ_λ is the learning rate for λ , $step$ is the training iteration number.

4. Experimental Results. We quantify the performance of regularized softmax on Unet and Segnet using Caffe implementation. Since Unet is prominent in biomedical image segmentation, Unet and Unet with regularized softmax activation function (RUnet) are tested on White Blood Cell Dataset. Segnet and Segnet with regularized softmax activation function (RSegnet) are tested on CamVid Dataset and SUN-RGBD Dataset.

We use SGD solver with momentum of 0.9 for each network. The learning rates of Unet and RUnet are fixed to be 0.0001, their weights are both randomly initialized. The learning rates of Segnet and RSegnet are fixed to be 0.001, their weights are both initialized from the VGG model trained on ImageNet using the techniques described in He et al.[11], the same as the author of Segnet did.

During testing stage, we first train those four networks on clean training dataset and test them on both clean and noisy images in order to further evaluate the robustness of our proposed method when encountering noise. Adding noise to training images is a common technique to make networks robust to noise, we also train the four networks on noisy data to make further comparison. We choose global accuracy and mean intersection over union (mIoU) to be our quantitative measures.

Given a segmentation result u , we evaluate its regularization effect as follows:

$$(4.1) \quad RE(u) = \frac{100}{N_1 \times N_2} \sum_{i=1}^{N_1} \sum_{j=1}^{N_2} |\nabla u_{i,j}|$$

where N_1, N_2 are the width and height of u , respectively. ∇u is defined as follows:

$$(4.2) \quad (\nabla u)_{i,j} = ((\nabla u)_{i,j}^1, (\nabla u)_{i,j}^2).$$

Segmentation results with higher RE means lower regularization effect, which often have more isolated small regions and serrated edges.

4.1. WBC Dataset. White Blood Cell Image Dataset[33] consists of two sub-datasets. Dataset 1 contains three hundred 120x120 images and their color depth is 24 bits. Dataset 2 consists of one hundred 300x300 color images. The cell images are generally purple and may contain many red blood cells around the white blood cells. Since the image size of Dataset 1 is a little bit small, it is not suitable for deep CNNs like Unet. We select Dataset 2 as our experimental dataset.

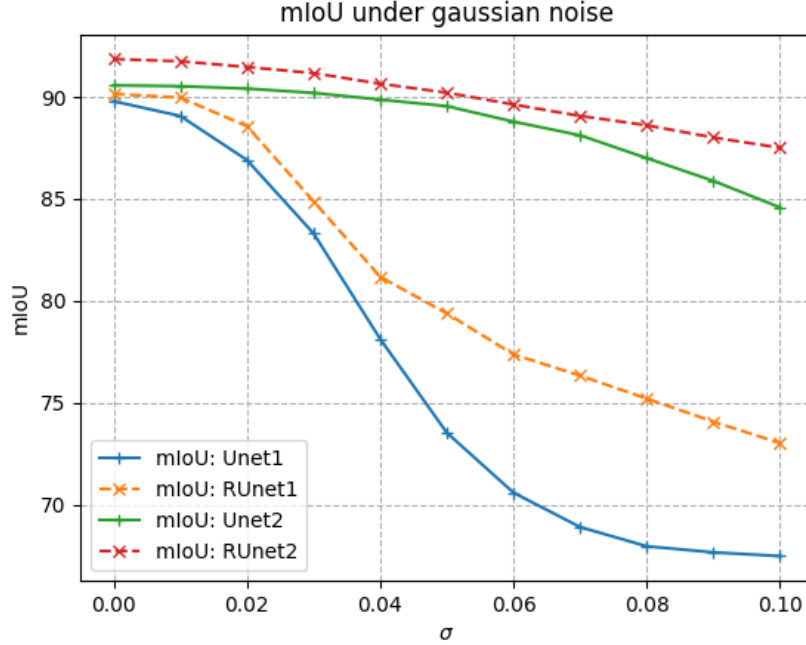


Figure 2. *Unet1 and RUnet1 are trained on clean WBC dataset, Unet2 and RUnet2 are trained on noisy WBC dataset. We add gaussian noise with zero mean, standard deviation σ from 0.01 to 0.1 to WBC testing dataset.*

WBC Dataset 2 has simple image structure and distinct details, it is very convenient for us to observe the difference in details intuitively. We replace original softmax layer with regularized softmax layer, other layers and parameters of Unet remain the same.

We randomly pick out 60 images from Dataset 2 as training dataset, the other 40 images are used for testing. Both Unet and RUnet are trained for 20k iterations, their mini-batch sizes are both eight.

As we cannot always obtain very clean images in practice, we want to know how the segmentation result will change when encountering small noise. First, Unet, RUnet, Segnet and RSegnet are all trained on clean data and tested on both clean and noise data with different noise. In Figure 2 we can see that mIoU of Unet1 has a significant drop when the noise level increases. However, the degradation in mIoU of RUnet1 is greatly alleviated. The benefit in mIoU from regularized softmax layer is very impressive. The mIoU curve of Unet1 seems to be convergent when σ is greater than 0.8 because of the majority pixels are recognized

Table 1*Results of Unet1 and RUnet1 trained on clean dataset.*

	Method	clean		gaussain				pepper	salt
noise level			0.01	0.03	0.05	0.07	0.09	0.01	0.01
mIoU	Unet1	89.79	89.02	83.19	73.94	68.74	67.67	80.68	77.75
	RUnet1	90.15	90.01	84.95	79.25	76.14	74.20	85.46	84.07
Accuracy	Unet1	97.04	96.69	93.55	86.43	81.22	80.33	92.55	90.00
	RUnet1	97.13	97.04	94.35	90.54	88.06	86.55	94.87	96.26
RE	Unet1	1.82	1.94	3.50	7.80	11.07	13.60	5.99	4.39
	RUnet1	1.30	1.30	1.39	1.48	1.52	1.56	1.35	1.32

Table 2*Results of Unet2 and RUnet2 trained on noisy dataset.*

	Method	clean		gaussain				pepper	salt
noise level			0.01	0.03	0.05	0.07	0.09	0.01	0.01
mIoU	Unet2	90.57	90.52	90.31	89.49	88.07	85.87	88.20	88.84
	RUnet2	91.86	91.73	91.19	90.24	89.29	88.37	89.85	91.22
Accuracy	Unet2	97.25	97.23	97.14	96.89	96.47	95.77	96.54	96.77
	RUnet2	97.64	97.60	97.38	97.00	96.67	96.38	96.96	97.37
RE	Unet2	1.79	1.80	1.87	2.02	2.17	2.87	2.38	2.19
	RUnet2	1.32	1.32	1.33	1.35	1.35	1.34	1.32	1.34

as background.

When the training dataset contains images with noise, the trained model can be more robust. We randomly pick out 20 images in training dataset and randomly add gaussian noise with zero mean, $\sigma = 0.05$ or pepper and salt noise with 1% pixels' value changed to each image. We make a further comparison when Unet and RUnet are trained on noisy dataset. Table 1 and Table 2 show predictions on clean data and data with different noise levels: gaussian noise with zero mean, standard deviations $\sigma = 0.01, 0.03, 0.05, 0.07, 0.09$, pepper and salt noise with 1% pixels' value changed per image.

In Figure 2 we can see that the loss of performance is greatly reduced when adding some noise to training images. In Table 2, performance of RUnet model is still better than Unet model. We also apply post-TV processing to segmentation results of Unet, that is we replace softmax with regularized softmax during testing stage and perform Equation (3.8) 100 iterations for each prediction. In Figure 3 row 4, we can see that the segmentation results have little improvement after post-TV processing, and it is still inferior to RUnet. This shows that our proposed regularized softmax help Unet find a better local optimum. Although post-TV processing could also bring regularization effect, it doesn't contribute to updating the model weights during training stage and its λ is manually set and thus not learnable, over-

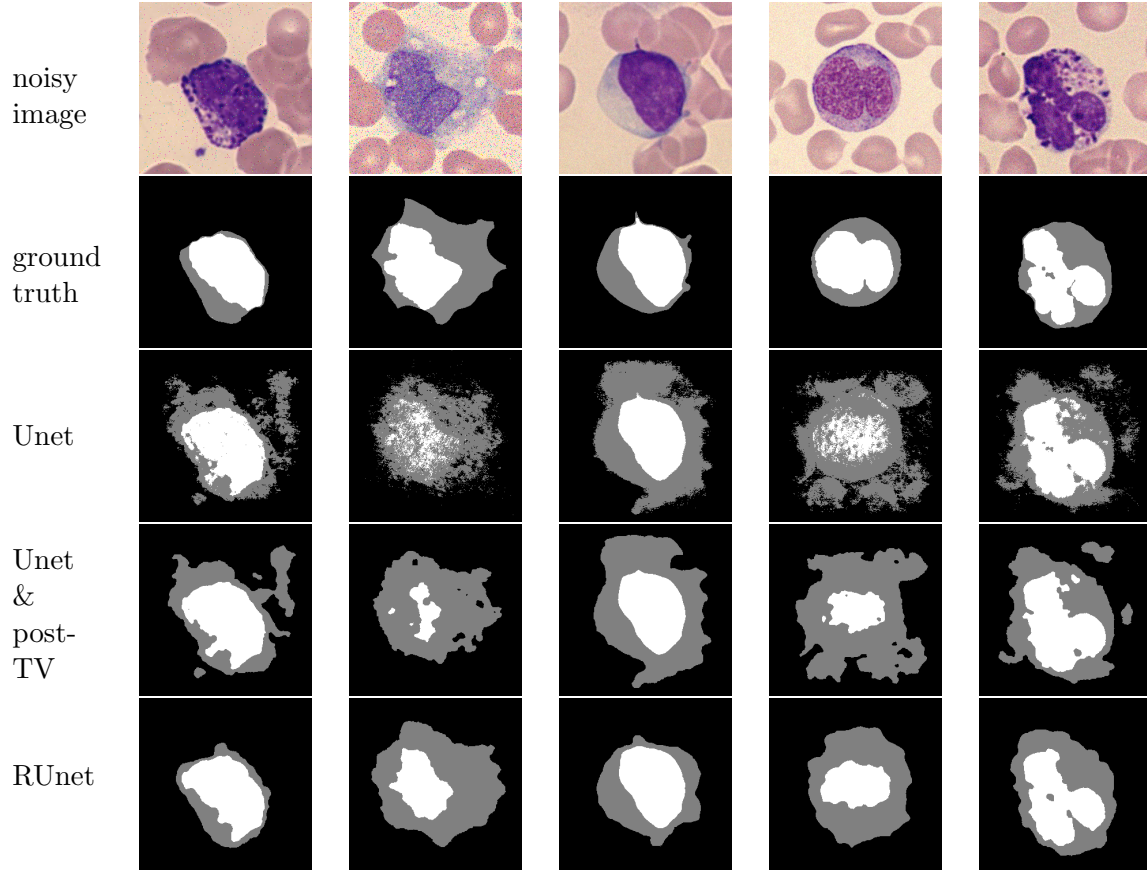


Figure 3. Segmentation results predicted by Unet and RUnet trained on noisy dataset. Noise type from left to right: small level salt and pepper(s&p) noise, large level s&p noise, small level gaussian noise, medium level gaussian noise, medium level gaussian noise.

regularization may happen. Our trainable λ scheme helps avoid falling into such a problem.

We can see obvious degradation in predictions on noisy images from row 3 in Figure 3. However, regularized softmax alleviates this problem, the segmentation results of RUnet are much better. We also observe that edges in segmentation results provided by RUnet are very smooth and there are no isolated points.

We have tried to select a best λ for post-TV processing and there are much less isolated points and regions in Figure 3 row 4. But over-regularization also happens due to manually set λ . In Figure 3 column 5, we can see that there are small holes inside the nucleus in ground truth. However, after applying post-TV processing, those holes just disappears in segmentation result at row 4 due to over-regularization. The segmentation result at row 5 preserves this detail.

4.2. CamVid Dataset. CamVid Dataset[13] is a collection of videos with object class semantic labels. This sequence depicts a moving driving scene in the city of Cambridge filmed from a moving car. It is a challenging dataset and selected as the benchmark dataset of

Segnet. The authors of Segnet use an 11 class version with an image size of 360 by 480, and they pick out 367 frames as training images and 233 frames as testing images.

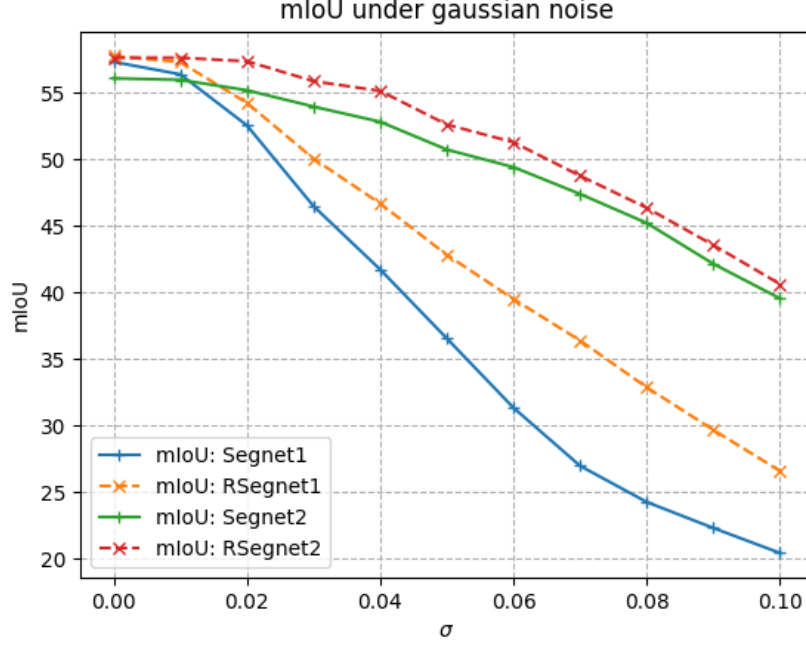


Figure 4. Segnet1 and RSegnet1 are trained on clean CamVid dataset, Segnet2 and RSegnet2 are trained on noisy CamVid dataset. We add gaussian noise with zero mean, standard deviation σ from 0.01 to 0.1 to CamVid testing dataset.

We replace original softmax layer with regularized softmax layer, other layers and parameters of Segnet and RSegnet remain the same. Both Segnet and RSegnet are trained for 80k iterations with weights initialized from the VGG model trained on ImageNet, their mini-batch sizes are both four. Learning rates of Segnet and RSegnet are fixed to be 0.001.

Table 3
Results of Segnet1 and RSegnet1 trained on clean dataset.

Method		clean	gaussain					pepper	salt
noise level			0.01	0.03	0.05	0.07	0.09	0.01	0.01
mIoU	Segnet1	57.35	56.42	46.51	36.57	26.99	22.32	50.42	35.45
	RSegnet1	57.79	57.34	50.08	42.85	36.41	29.69	52.80	41.16
Accuracy	Segnet1	87.74	87.42	81.71	71.27	55.82	48.92	81.52	71.33
	RSegnet1	88.01	87.74	84.18	78.44	72.20	63.12	84.17	78.85
RE	Segnet1	4.10	4.18	5.62	7.79	9.00	9.63	5.88	7.04
	RSegnet1	2.43	2.44	2.69	3.20	3.78	4.27	2.69	3.11

Table 4
Results of Segnet2 and RSegnet2 trained on noisy dataset.

	Method	clean		gaussain				pepper	salt
noise level			0.01	0.03	0.05	0.07	0.09	0.01	0.01
mIoU	Segnet2	56.12	56.00	53.99	50.76	47.42	42.17	55.54	50.82
	RSegnet2	57.66	57.66	55.90	52.64	48.79	43.58	57.21	51.81
Accuracy	Segnet2	87.54	87.52	86.95	85.56	83.43	79.27	87.22	85.90
	RSegnet2	88.10	88.09	87.63	86.12	83.56	79.36	87.68	86.05
RE	Segnet2	4.56	4.52	4.57	4.82	5.41	6.60	4.51	4.71
	RSegnet2	2.51	2.49	2.49	2.59	2.93	3.39	2.46	2.50

First, Segnet and RSegnet are all trained on clean data and tested on data with different noise. Similar degradation could be found in Figure 4. In Figure 5 we can see the prediction results of bicyclist and road become very messy in segmentation results predicted by Segnet at column 1 and column 4, whereas RSegnet provides relative good segmentation results.

Then, we randomly pick out 90 images in training dataset and randomly add gaussian noise with zero mean, $\sigma = 0.03$ or pepper and salt noise with 1% pixels' value changed to each image. We make a further comparison when Segnet and RSegnet are trained on noisy dataset. Table 3 and Table 4 show predictions on data with different noise levels. In Figure 4, we can find that the performance of Segnet and RSegnet on clean testing dataset has a little drop when trained on noisy data, this is due to the different data distribution between training data and testing data. Road scene has very complex image structure, adding noise to training dataset brings further complexity to training task. However, the accuracy, mIoU of Segnet model and RSegnet model appear much more robust to noise. This is a trade-off between accuracy and robustness.

The performance of RSegnet is better than Segnet. Although post-TV processing brings regularization effect to the segmentation results of Segnet, over-regularization also happens. In Figure 5 column 3, we can see that the distant column-pole is over-regularized after post-TV processing. Nevertheless, it is well preserved by RSegnet.

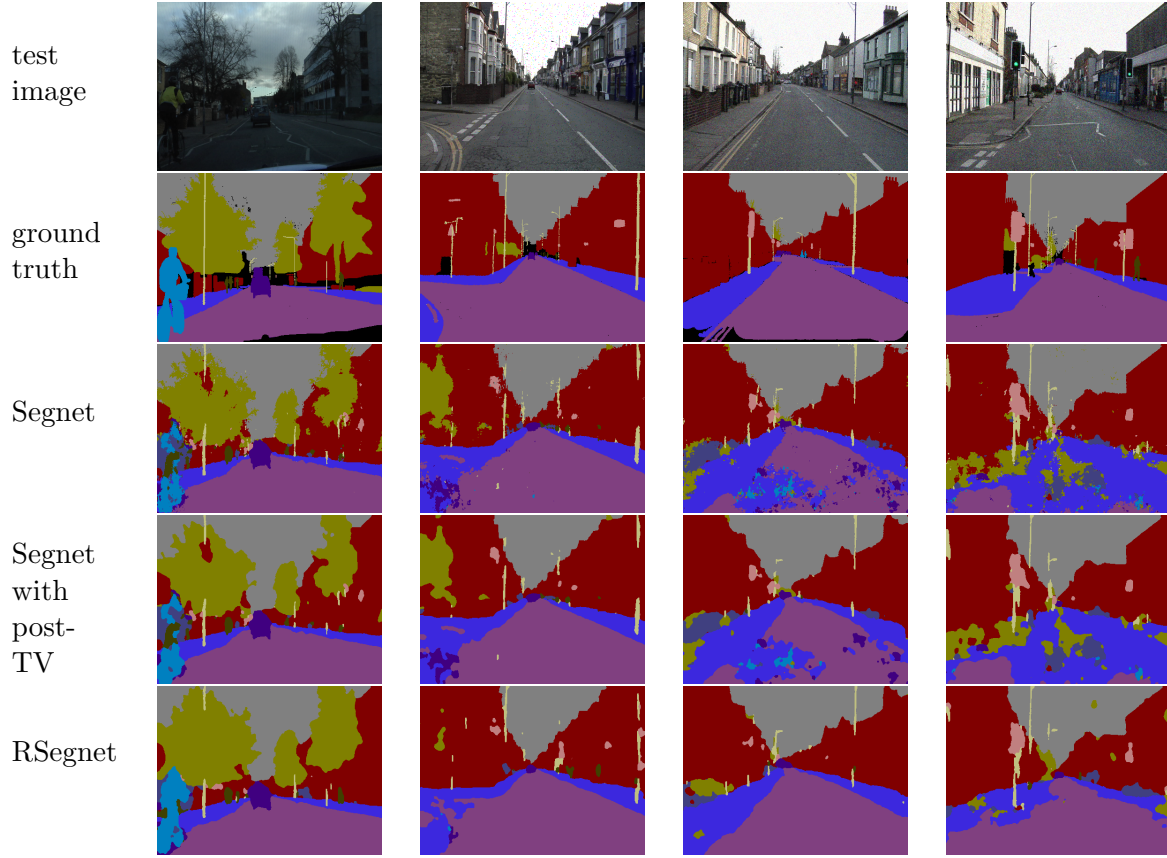


Figure 5. Segmentation results of Segnet and RSegnet trained on noisy dataset. Noise type from left to right: clean image, medium level pepper noise, medium level gaussian noise, large level gaussian noise.

4.3. SUN-RGBD Dataset. SUN-RGBD Dataset[28] is a much more challenging dataset of indoor scenes with 10355 images in total. We randomly select 5,285 images as our training dataset and the remaining images are used as testing dataset. The annotation files contain thousands of labels and we select 37 main categories as our segmentation classes, like the authors of Segnet did. Different sensors are used to capture the scenes and images are in various resolutions. A stochastic patch of size 360 by 480 is cropped from each image.

We replace original softmax layer with regularized softmax layer, other layers and parameters of Segnet and RSegnet remain the same. Both Segnet and RSegnet are trained for 200k iterations with weights initialized from the VGG model trained on ImageNet, their mini-batch sizes are both three. Learning rates of Segnet and RSegnet are fixed to be 0.001.

First, Segnet and RSegnet are all trained on clean data and tested on data with different noise. Then, we randomly pick out 1000 images in training dataset and randomly add gaussian noise with zero mean, $\sigma = 0.05$ or pepper and salt noise with 1% pixels' value changed to each image. We make a further comparison when Segnet and RSegnet are trained on noisy dataset. Table 5 and Table 6 show predictions on data with different noise levels.

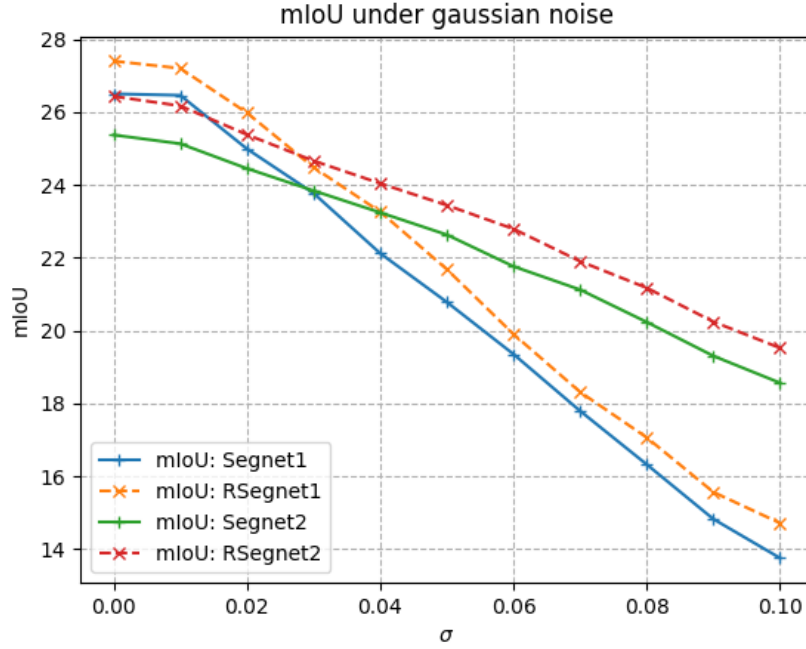


Figure 6. *Segnet1 and RSegnet1 are trained on clean SUN-RGBD Dataset, Segnet2 and RSegnet2 are trained on noisy SUN-RGBD dataset. We add gaussian noise with zero mean, standard deviation σ from 0.01 to 0.1 to SUN-RGBD testing dataset.*

Table 5

Results of Segnet1 and RSegnet1 trained on clean dataset.

	Method	clean		gaussain				pepper	salt
noise level		0.01	0.03	0.05	0.07	0.09	0.01	0.01	
mIoU	Segnet1	26.50	26.46	23.77	20.78	17.79	14.82	19.79	19.28
	RSegnet1	27.40	27.20	24.48	21.68	18.31	15.56	21.01	20.68
Accuracy	Segnet1	68.62	68.42	66.75	64.26	61.40	58.03	63.00	60.53
	RSegnet1	68.94	68.70	66.83	64.49	61.68	58.82	63.22	60.89
RE	Segnet1	4.99	5.01	4.99	5.01	5.08	5.24	5.64	5.54
	RSegnet1	2.74	2.73	2.72	2.69	2.71	2.72	2.90	2.89

Since images in SUN-RGBD Dataset are captured by different sensors with different resolutions, quality of images are uneven. Compared to WBC and CamVid datasets, SUN-RGBD dataset is not so clean and tidy. What's more, thousands labels appear in the original annotation file of SUN-RGBD Dataset, resulting in a much more complex image structure. All these greatly increase the difficulty of the learning task. When introducing the same level noise, the mIoU curves of Segnet and RSegnet are closer than those in the other two datasets due to

Table 6
Results of Segnet2 and RSegnet2 trained on noisy dataset.

	Method	clean		gaussain				pepper	salt
noise level		0.01	0.03	0.05	0.07	0.09	0.01	0.01	
mIoU	Segnet2	25.37	25.13	23.84	22.63	21.12	19.30	23.25	23.36
	RSegnet2	26.43	26.17	24.66	23.44	21.90	20.24	24.37	24.42
Accuracy	Segnet2	67.52	67.36	66.36	65.14	63.61	61.85	65.89	65.29
	RSegnet2	68.29	68.08	66.97	65.72	64.36	62.94	66.73	66.19
RE	Segnet2	4.84	4.84	4.91	5.03	5.21	5.40	5.03	5.10
	RSegnet2	2.31	2.49	2.49	2.59	2.93	3.39	2.35	2.36

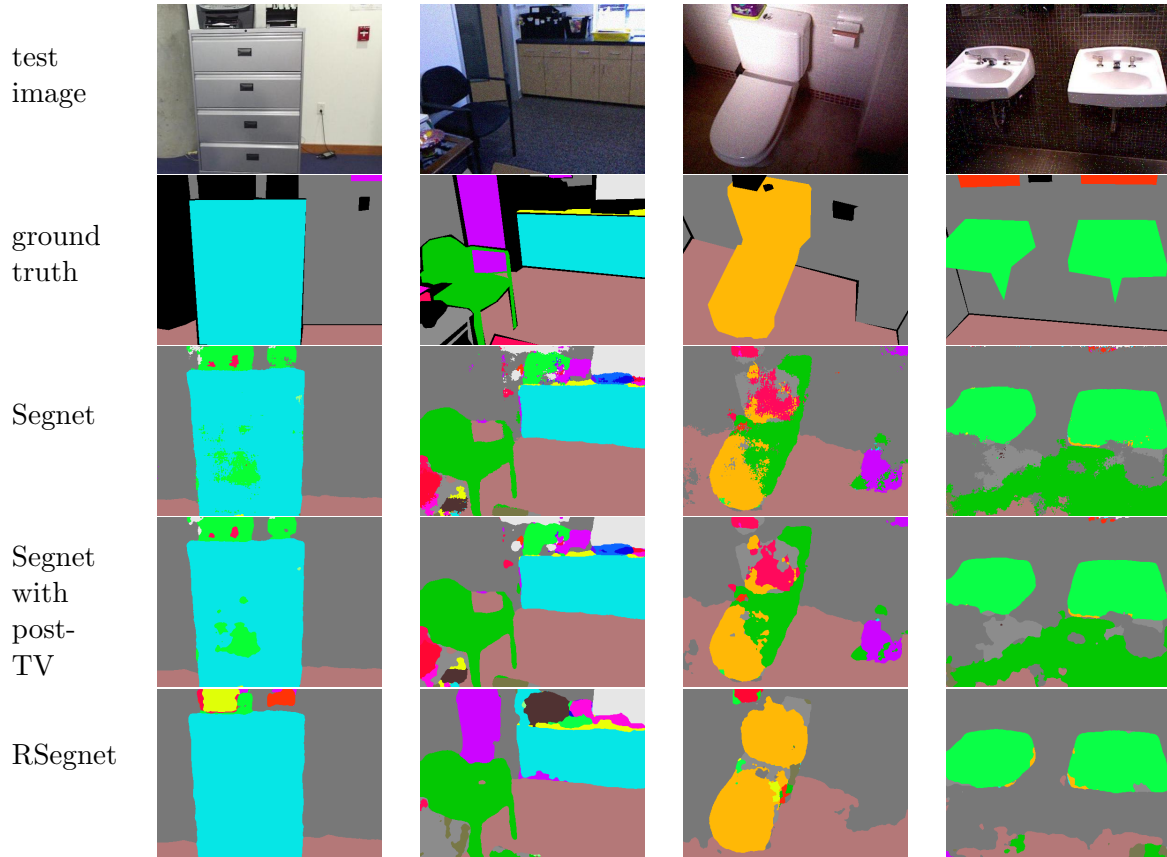


Figure 7. Segmentation results of Segnet and RSegnet trained on clean dataset. Noise type from left to right: clean image, medium level gaussian noise, medium level gaussian noise, small level salt noise.

latent perturbation has been added to SUN-RGBD dataset. Nevertheless, RSegnet still shows better performance than Segnet. It seems that we can benefit more from regularized softmax when it is applied on clean and tidy dataset.

5. Conclusions and Future Work. Motivated by the desire for obtaining regularized edges, eliminating scattered points and tiny regions, we propose regularized softmax on CNNs for semantic image segmentation. By applying our method to regularizing Unet and Segnet, we observed better performance from experiments on WBC Dataset, CamVid Dataset and SUN-RGBD Dataset. The proposed method can be applied easily to many other CNNs and tasks. In the future, we will explore the potential to design an end-to-end network by training CNN layers and η simultaneously..

REFERENCES

- [1] V. BADRINARAYANAN, A. KENDALL, AND R. CIPOLLA, *Segnet: A deep convolutional encoder-decoder architecture for image segmentation*, arXiv preprint arXiv:1511.00561, (2015).
- [2] L. BARGHOUT AND L. LEE, *Perceptual information processing system*, Mar. 25 2004. US Patent App. 10/618,543.
- [3] G. J. BROSTOW, J. SHOTTON, J. FAUQUEUR, AND R. CIPOLLA, *Segmentation and recognition using structure from motion point clouds*, in European conference on computer vision, Springer, 2008, pp. 44–57.
- [4] A. CHAMBOLLE, *An algorithm for total variation minimization and applications*, Journal of Mathematical imaging and vision, 20 (2004), pp. 89–97.
- [5] A. CHAMBOLLE AND P.-L. LIONS, *Image recovery via total variation minimization and related problems*, Numerische Mathematik, 76 (1997), pp. 167–188.
- [6] L.-C. CHEN, G. PAPANDREOU, I. KOKKINOS, K. MURPHY, AND A. L. YUILLE, *Semantic image segmentation with deep convolutional nets and fully connected crfs*, arXiv preprint arXiv:1412.7062, (2014).
- [7] L.-C. CHEN, G. PAPANDREOU, I. KOKKINOS, K. MURPHY, AND A. L. YUILLE, *Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs*, IEEE transactions on pattern analysis and machine intelligence, 40 (2018), pp. 834–848.
- [8] D. ERHAN, C. SZEGEDY, A. TOSHEV, AND D. ANGUELOV, *Scalable object detection using deep neural networks*, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2014, pp. 2147–2154.
- [9] R. GIRSHICK, J. DONAHUE, T. DARRELL, AND J. MALIK, *Rich feature hierarchies for accurate object detection and semantic segmentation*, in Proceedings of the IEEE conference on computer vision and pattern recognition, 2014, pp. 580–587.
- [10] B. HARIHARAN, P. ARBELÁEZ, R. GIRSHICK, AND J. MALIK, *Simultaneous detection and segmentation*, in European Conference on Computer Vision, Springer, 2014, pp. 297–312.
- [11] K. HE, X. ZHANG, S. REN, AND J. SUN, *Delving deep into rectifiers: Surpassing human-level performance on imagenet classification*, in Proceedings of the IEEE international conference on computer vision, 2015, pp. 1026–1034.
- [12] K. HE, X. ZHANG, S. REN, AND J. SUN, *Deep residual learning for image recognition*, in Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 770–778.
- [13] M. JOHNSON-ROBERSON, C. BARTO, R. MEHTA, S. N. SRIDHAR, K. ROSAEN, AND R. VASUDEVAN, *Driving in the matrix: Can virtual worlds replace human-generated annotations for real world tasks?*, arXiv preprint arXiv:1610.01983, (2016).
- [14] P. KRÄHENBÜHL AND V. KOLTUN, *Efficient inference in fully connected crfs with gaussian edge potentials*, in Advances in neural information processing systems, 2011, pp. 109–117.
- [15] A. KRIZHEVSKY, I. SUTSKEVER, AND G. E. HINTON, *Imagenet classification with deep convolutional neural networks*, in Advances in neural information processing systems, 2012, pp. 1097–1105.
- [16] L. LADICKÝ, P. STURGESS, K. ALAHARI, C. RUSSELL, AND P. H. TORR, *What, where and how many?*

- combining object detectors and crfs, in European conference on computer vision, Springer, 2010, pp. 424–437.
- [17] Y. LECUN, L. BOTTOU, Y. BENGIO, AND P. HAFFNER, *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, 86 (1998), pp. 2278–2324.
 - [18] W. LIU, D. ANGUELOV, D. ERHAN, C. SZEGEDY, S. REED, C.-Y. FU, AND A. C. BERG, *Ssd: Single shot multibox detector*, in European conference on computer vision, Springer, 2016, pp. 21–37.
 - [19] J. LONG, E. SHELHAMER, AND T. DARRELL, *Fully convolutional networks for semantic segmentation*, in Proceedings of the IEEE conference on computer vision and pattern recognition, 2015, pp. 3431–3440.
 - [20] H. NOH, S. HONG, AND B. HAN, *Learning deconvolution network for semantic segmentation*, in Proceedings of the IEEE international conference on computer vision, 2015, pp. 1520–1528.
 - [21] P. OCHS, R. RANFTL, T. BROX, AND T. POCK, *Techniques for gradient-based bilevel optimization with non-smooth lower level problems*, Journal of Mathematical Imaging and Vision, 56 (2016), pp. 175–194.
 - [22] G. PAPANDREOU, I. KOKKINOS, AND P.-A. SAVALLE, *Modeling local and global deformations in deep learning: Epitomic convolution, multiple instance learning, and sliding window detection*, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 390–399.
 - [23] O. RONNEBERGER, P. FISCHER, AND T. BROX, *U-net: Convolutional networks for biomedical image segmentation*, in International Conference on Medical image computing and computer-assisted intervention, Springer, 2015, pp. 234–241.
 - [24] L. I. RUDIN, S. OSHER, AND E. FATEMI, *Nonlinear total variation based noise removal algorithms*, Physica D: nonlinear phenomena, 60 (1992), pp. 259–268.
 - [25] P. SERMANET, D. EIGEN, X. ZHANG, M. MATHIEU, R. FERGUS, AND Y. LECUN, *Overfeat: Integrated recognition, localization and detection using convolutional networks*, arXiv preprint arXiv:1312.6229, (2013).
 - [26] L. SHAPIRO AND G. C. STOCKMAN, *Computer vision. 2001*, ed: Prentice Hall, (2001).
 - [27] J. SHOTTON, M. JOHNSON, AND R. CIPOLLA, *Semantic texton forests for image categorization and segmentation*, in Computer vision and pattern recognition, 2008. CVPR 2008. IEEE Conference on, IEEE, 2008, pp. 1–8.
 - [28] S. SONG, S. P. LICHTENBERG, AND J. XIAO, *Sun rgb-d: A rgb-d scene understanding benchmark suite*, in Proceedings of the IEEE conference on computer vision and pattern recognition, 2015, pp. 567–576.
 - [29] P. STURGESS, K. ALAHARI, L. LADICKY, AND P. H. TORR, *Combining appearance and structure from motion features for road scene understanding*, in BMVC-British Machine Vision Conference, BMVA, 2009.
 - [30] C. WU AND X.-C. TAI, *Augmented lagrangian method, dual methods, and split bregman iteration for rof, vectorial tv, and high order models*, SIAM Journal on Imaging Sciences, 3 (2010), pp. 300–339.
 - [31] M. D. ZEILER AND R. FERGUS, *Visualizing and understanding convolutional networks*, in European conference on computer vision, Springer, 2014, pp. 818–833.
 - [32] H. ZHAO, J. SHI, X. QI, X. WANG, AND J. JIA, *Pyramid scene parsing network*, in IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 2881–2890.
 - [33] X. ZHENG, Y. WANG, G. WANG, AND J. LIU, *Fast and robust segmentation of white blood cell images by self-supervised learning*, Micron, 107 (2018), pp. 55–71, <https://doi.org/https://doi.org/10.1016/j.micron.2018.01.010>, <https://www.sciencedirect.com/science/article/pii/S0968432817303037>.